

CURASCAN

**Plateforme Multi-Agents de Pré-diagnostic Médical
avec Validation Humaine (HITL)**

Rapport de Projet de Fin d'Année

Spécialité : IA & Data

Réalisé par :

Asmaa & Abir

(Product Owner & Full-Stack Developer)

Année Académique : 2026

1 Introduction & Contexte du Projet

1.1 1.1 Introduction

Le secteur de la santé fait face à une augmentation constante de la charge de travail des professionnels médicaux. **CuraScan** est une solution innovante qui intègre l'Intelligence Artificielle au cœur du parcours de soin, agissant comme un assistant augmenté aux côtés du médecin sans jamais s'y substituer.

1.2 1.2 Problématique

L'intégration de l'IA en médecine se heurte souvent à plusieurs défis majeurs :

- **Surcharge cognitive** : Collecte fastidieuse des symptômes initiaux lors des consultations.
- **Effet boîte noire** : Manque de transparence et de confiance du corps médical envers les diagnostics 100% automatisés.
- **Risques cliniques** : Possibilité d'erreurs ou d'omissions sous la pression des flux de patients.

1.3 1.3 Objectifs du Projet

- Développer un système interactif capable de poser des questions ciblées et adaptées au patient.
- Garantir la sécurité médicale grâce à un workflow *Human-in-the-Loop* (HITL).
- Automatiser la génération de rapports cliniques exportables et structurés.

2 Architecture Technique & Pipeline Multi-Agents

2.1 2.1 Architecture Globale du Système

Le projet s'appuie sur une architecture découplée assurant modularité, évolutivité et performances optimales :

- **Frontend (React / Vite)** : Interface utilisateur moderne et réactive développée en TypeScript, utilisant TailwindCSS et les composants Shadcn. Elle gère l'affichage dynamique des phases du workflow.
- **Backend (FastAPI / Python)** : API asynchrone haute performance chargée d'orchestrer les modèles et de superviser le cycle de vie des agents.

2.2 2.2 Orchestration Multi-Agents avec LangGraph

Pour formaliser la logique métier et gérer l'interaction asynchrone, nous utilisons **LangGraph** pour implémenter un graphe orienté d'agents spécialisés :

Listing 1 – Structure du Backend FastAPI (main.py)

```
@app.post("/api/chat")
async def chat_endpoint(req: ChatRequest):
    config = {"configurable": {"thread_id": req.thread_id}}
    input_data = {"messages": [{"role": "user", "content": req.message}]}

    async for event in app_with_memory.astream(input_data, config):
        pass

    final_state = app_with_memory.get_state(config)
    return {"response": final_state.values.get("messages")[-1].content}
```

2.3 2.3 Le Pipeline des Agents

Le cycle de traitement est divisé en trois phases distinctes :

1. **Diagnostic Agent (Phase active)** : Recueille les symptômes et pose un maximum de 5 questions chirurgicales.
2. **Revue Médecin (Phase HITL / Breakpoint)** : Interrompt le graphe. L'IA passe en attente d'une validation ou correction humaine.
3. **Report Agent (Phase finale)** : Consolide l'historique et rédige de façon autonome le résumé clinique et les conseils de surveillance.

3 Expérimentations, Résultats & Interfaces

3.1 3.1 Implémentation Technologique

- **Modèles de Langage (LLM)** : Utilisation d'architectures LLM avancées connectées via un système de mémoire persistant (`app_with_memory`) pour conserver le contexte exact de chaque patient grâce au paramètre `thread_id`.
- **Gestion des États** : Utilisation de structures Pydantic strictes (`MedicalState`) pour valider les données qui transitent entre les agents.

3.2 3.2 Interfaces Clés du Projet

L'interface utilisateur reflète précisément l'état d'avancement du graphe en temps réel :

- **Composant Workflow** : Un fil d'ariane dynamique affichant les étapes : *Diagnostic en cours* → *Attente Médecin* → *Génération du rapport*.
- **Module de Validation (HITL)** : Une zone sécurisée permettant au médecin de relire la synthèse clinique, d'ajuster les diagnostics proposés et de lever le blocage du pipeline de l'IA en un clic.

3.3 3.3 Évaluation des Performances

Les tests d'intégration menés entre l'interface frontend et l'API asynchrone FastAPI démontrent une excellente réactivité du système. L'état du graphe est correctement maintenu, et l'interruption au nœud de revue médicale bloque efficacement l'exécution jusqu'à l'action explicite du praticien, écartant tout risque d'erreur automatisée.

4 Conclusion & Perspectives

4.1 Bilan du Projet

CuraScan démontre avec succès la viabilité et la pertinence des architectures multi-agents appliquées au domaine de la e-santé. L'utilisation conjointe de FastAPI et de LangGraph offre une séparation claire des responsabilités algorithmiques et garantit une traçabilité complète des décisions.

Le concept fondamental de validation humaine (*Human-in-the-Loop*) permet de lever le frein majeur de l'adoption de l'IA en médecine, en remplaçant le praticien au centre du contrôle tout en l'allégeant des tâches de rédaction répétitives.

4.2 Perspectives d'Évolution

Plusieurs axes d'amélioration ont été identifiés pour la suite du développement du projet :

- **Spécialisation des agents** : Intégrer des sous-agents experts par spécialité médicale (ex : cardiologie, pédiatrie, dermatologie).
- **Analyse d'imagerie médicale** : Connecter le système à des modèles de vision (*Computer Vision*) pour analyser directement des radiographies ou des scanners en complément des symptômes textuels.
- **Sécurisation et Déploiement Cloud** : Conteneuriser l'application complète via Docker et orchestrer son déploiement sur une infrastructure Cloud certifiée HDS (Hébergement de Données de Santé).